

Microsoft Fabric Eventhouse에서 데이터 작업하기

Microsoft Fabric에서 `eventhouse`는 실시간 이벤트 관련 데이터를 저장하는 데 사용되며, 종종 `eventstream`을 통해 스트리밍 데이터 소스에서 데이터를 캡처합니다.

개념 설명: Eventhouse, KQL Database, Eventstream 이란?

- **Eventhouse:** Microsoft Fabric의 실시간 인텔리전스(Real-Time Intelligence) 환경에서 중심적인 역할을 하는 데이터 저장소입니다. 이름에서 알 수 있듯이, '이벤트(Event)' 데이터, 즉 시간의 흐름에 따라 지속적으로 발생하는 데이터(예: IoT 센서 로그, 웹사이트 클릭 로그, 금융 거래 내역)를 저장, 관리, 분석하는 데 최적화되어 있습니다. Eventhouse는 대규모의 실시간 데이터를 효율적으로 처리하기 위한 엔진과 구조를 갖추고 있습니다.
- **KQL Database:** Eventhouse의 핵심 저장소는 바로 KQL Database입니다. 이것은 Azure Data Explorer(ADX)에서 사용하는 것과 동일한 강력한 분석 데이터베이스 엔진을 기반으로 합니다. KQL Database는 시계열 데이터(time-series data)와 비정형 데이터(예: 로그, 텍스트)를 매우 빠른 속도로 쿼리하고 분석하는 데 특화되어 있습니다. 데이터는 열(Column) 기반으로 저장되어 대규모 데이터셋에 대한 집계 및 분석 작업에서 뛰어난 성능을 보입니다.
- **Eventstream:** Eventstream은 실시간 데이터를 Fabric으로 가져오는 파이프라인 역할을 합니다. 다양한 소스(예: Azure Event Hubs, IoT Hub 등)로부터 들어오는 데이터 스트림을 캡처하여 Eventhouse 내의 KQL Database나 Lakehouse 같은 다른 대상으로 라우팅할 수 있습니다. 즉, Eventstream이 데이터의 '입구'라면, Eventhouse는 그 데이터가 저장되고 분석되는 '집'이라고 할 수 있습니다.

이 세 가지 요소는 함께 작동하여 데이터가 발생하는 순간부터 분석을 통해 인사이트를 얻기까지의 전체 과정을 원활하게 연결하는 엔드투엔드 실시간 분석 솔루션을 제공합니다.

Eventhouse 내에서 데이터는 하나 이상의 KQL Database에 저장되며, 각 데이터베이스에는 테이블 및 기타 객체들이 포함되어 있습니다. 이 데이터는 Kusto Query Language (KQL) 또는 일부 Structured Query Language (SQL)를 사용하여 쿼리할 수 있습니다.

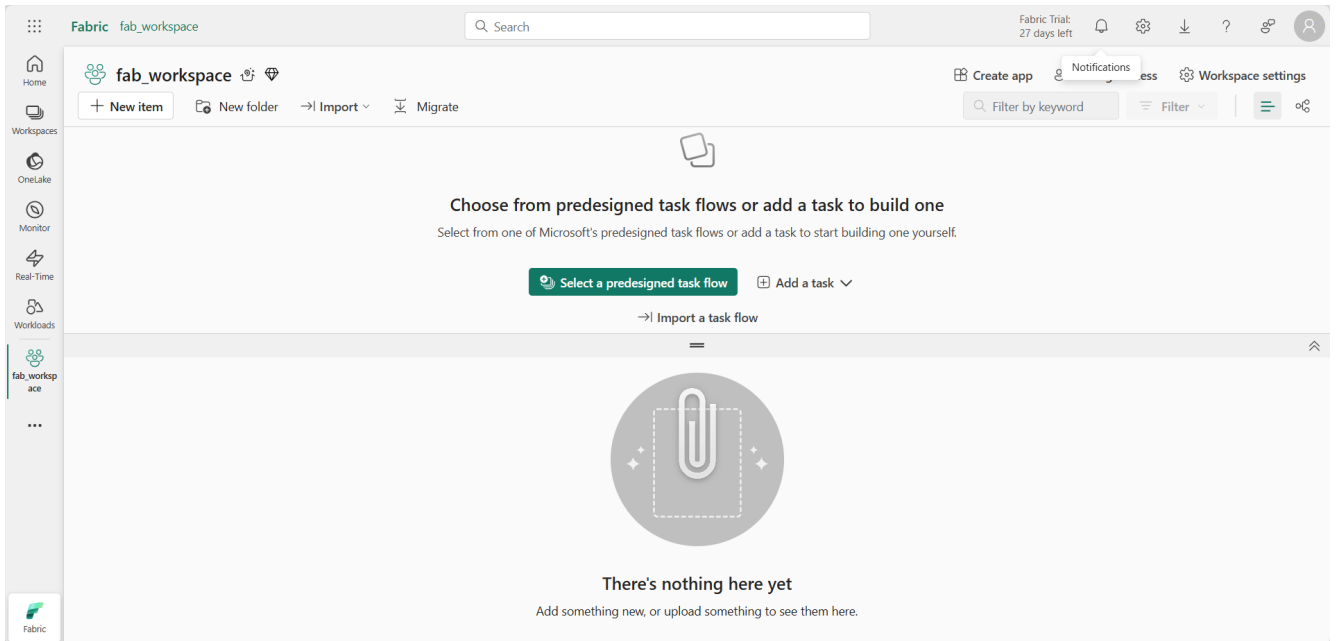
이 실습에서는 사전에 대여와 관련된 샘플 데이터를 사용하여 eventhouse를 생성하고 데이터를 채운 다음, KQL과 SQL을 사용하여 데이터를 쿼리합니다.

이 실습을 완료하는 데는 약 25분이 소요됩니다.

Workspace 생성

Fabric에서 데이터를 다루기 전에, Fabric 용량이 활성화된 Workspace(작업 영역)를 생성해야 합니다.

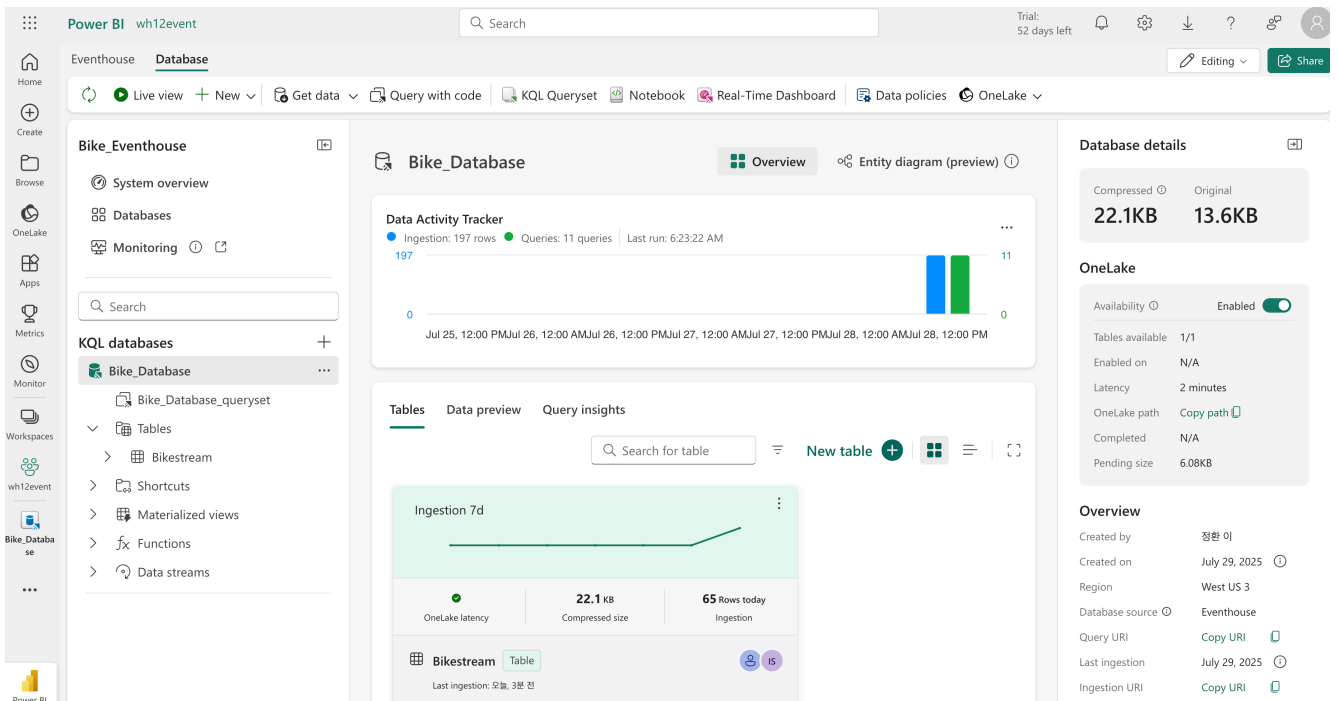
1. 브라우저에서 [Microsoft Fabric 홈페이지](https://app.fabric.microsoft.com/home?experience=fabric) (`https://app.fabric.microsoft.com/home?experience=fabric`)로 이동하여 Fabric 자격 증명으로 로그인합니다.
2. 왼쪽 메뉴 바에서 **Workspaces** (아이콘 모양 )를 선택합니다.
3. 원하는 이름으로 새 Workspace를 만들고, Fabric 용량을 포함하는 라이선스 모드(Trial, Premium, 또는 Fabric)를 선택합니다.
4. 새 Workspace가 열리면 비어 있어야 합니다.

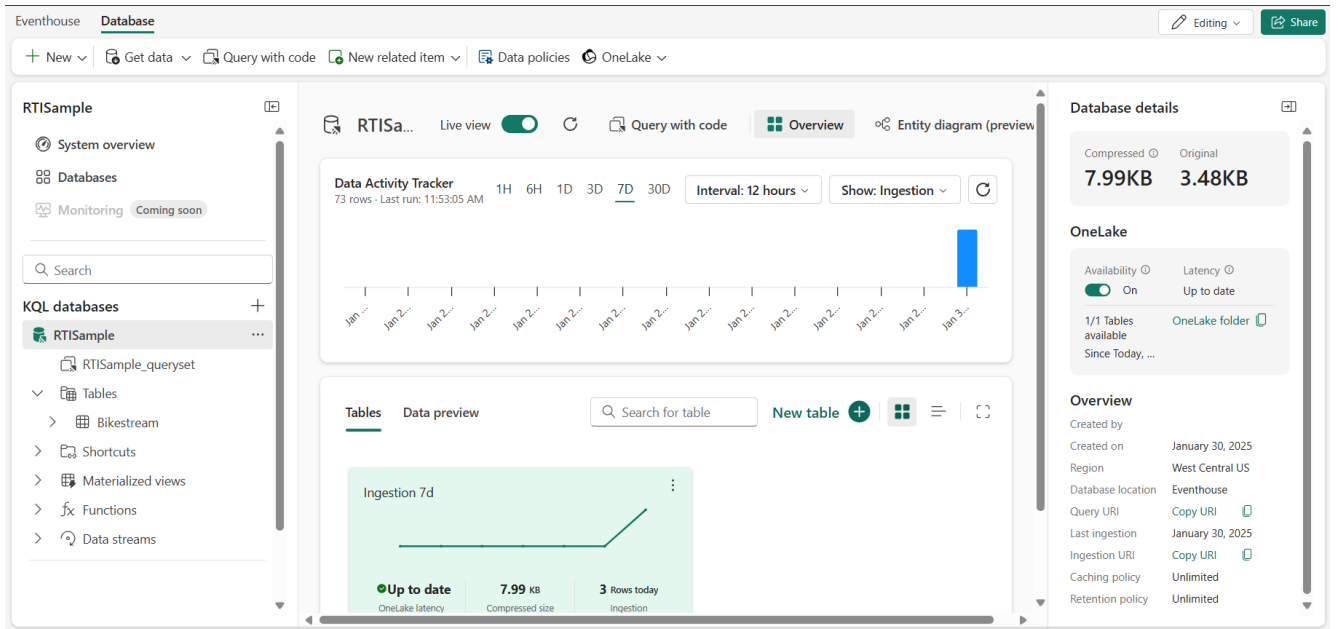


Eventhouse 생성

이제 Fabric 용량을 지원하는 Workspace가 있으므로 그 안에 `eventhouse` 를 생성할 수 있습니다.

1. 왼쪽 메뉴 바에서 **Workloads**를 선택한 다음, **Real-Time Intelligence** 타일을 선택합니다.
2. **Real-Time Intelligence** 홈페이지에서 **Explore Real-Time Intelligence Sample** 타일을 선택합니다. 그러면 **RTISample**이라는 `eventhouse` 가 자동으로 생성됩니다.





3. 왼쪽 창에서, `eventhouse` 와 동일한 이름의 `KQL Database` 가 포함되어 있는지 확인합니다.

4. **Bikestream** 테이블도 생성되었는지 확인합니다.

KQL을 사용하여 데이터 쿼리하기

개념 설명: Kusto Query Language (KQL)란?

KQL은 로그, 원격 측정 데이터, 시계열 데이터와 같은 대규모 데이터셋을 탐색하고 분석하기 위해 설계된 강력하고 직관적인 쿼리 언어입니다. SQL과는 다른 접근 방식을 가집니다.

- **파이프라인 기반 구문:** KQL 쿼리는 데이터가 한 연산자에서 다음 연산자로 '흘러가는' 파이프라인 형태를 가집니다. 데이터 소스(테이블 이름)로 시작하여 파이프(|) 문자로 각 연산자를 연결합니다. 이 구조는 데이터를 단계별로 필터링하고 변형하는 과정을 쉽게 이해하고 작성할 수 있게 해줍니다.
- **읽기 전용:** KQL은 데이터 분석 및 조회를 위한 언어이며, 데이터를 수정(INSERT, UPDATE, DELETE)하는 기능은 없습니다.
- **성능:** 대규모의 정형 및 비정형 데이터를 매우 빠르게 스캔하고 집계하는 데 최적화되어 있습니다.

SQL에 익숙한 사용자도 KQL의 직관적인 파이프라인 구문에 금방 적응할 수 있으며, 특히 복잡한 데이터 분석 작업에서 그 진가를 발휘합니다.

KQL로 테이블에서 데이터 검색하기

1. `Eventhouse` 창의 왼쪽 창에서, `KQL Database` 아래에 있는 기본 **queryset** 파일을 선택합니다. 이 파일에는 시작에 도움이 되는 몇 가지 샘플 KQL 쿼리가 포함되어 있습니다.

Bike_Eventhouse

System overview

Databases

Monitoring ⓘ ↗

Search

KQL databases +

Bike_Database

Bike_Database_queryset

Tables

Bikestream

Shortcuts

Materialized views

Functions

Data streams

Tab

Run

Preview

Recall

Copy query

Pin to dashboard

</> K

```

5 //*****
6
7 // Use "take" to view a sample number of records in the table and check the data.
8 Bikestream
9 | take 100
10
11 // 'project'와 'take'를 사용하여 테이블의 샘플 레코드 수를 보고 데이터를 확인합니다.
12 Bikestream
13 | project Street, No_Bikes
14 | take 10
15
16
17 // See how many records are in the table.
18 Bikestream
19 | count
20

```

Table 1

+ Add visual

⦿ Stats

Search

🕒 2025-07-28

Street	No_Bikes
> Ormonde Gate	10
> Phene Street	21
> Limerston Street	18
> The Vale	13

2. 첫 번째 예제 쿼리를 다음과 같이 수정합니다.

```
Bikestream
| take 100
```

코드 설명:

- `Bikestream`: 쿼리를 시작할 테이블의 이름입니다. 데이터의 원천을 지정합니다.
- `|` (파이프): KQL의 핵심 연산자입니다. 왼쪽의 테이블 데이터(`Bikestream`)를 오른쪽의 연산자(`take`)로 전달하는 역할을 합니다. 마치 데이터가 흐르는 파이프와 같습니다.
- `take 100`: 테이블에서 임의의 100개 행(레코드)을 가져옵니다. 전체 데이터를 로드하지 않고 데이터의 구조나 내용을 빠르게 확인하고 싶을 때 매우 유용합니다.

3. 쿼리 코드를 선택하고 실행하여 테이블에서 100개의 행을 반환합니다.

`project` 키워드를 사용하여 쿼리하려는 특정 속성을 추가하고 `take` 키워드를 사용하여 엔진에 반환할 레코드 수를 알려 줌으로써 더 정확하게 지정할 수 있습니다.

4. 다음 쿼리를 입력하고 선택한 후 실행합니다.

```
// 'project'와 'take'를 사용하여 테이블의 샘플 레코드 수를 보고 데이터를 확인합니다.
Bikestream
| project Street, No_Bikes
| take 10
```

코드 설명:

- `//`: 한 줄 주석을 나타냅니다. 쿼리에 대한 설명을 추가할 때 사용됩니다.
- `project Street, No_Bikes`: 파이프를 전달된 데이터에서 `Street`와 `No_Bikes` 두 개의 열(Column)만 선택하여 결과에 포함시킵니다. SQL의 `SELECT` 절과 유사한 역할을 합니다.

- `take 10: project` 연산을 거친 결과에서 10개의 행을 가져옵니다.

분석에서 또 다른 일반적인 관행은 `queryset`의 열 이름을 더 사용자 친화적으로 바꾸는 것입니다.

5. 다음 쿼리를 시도해 보세요.

```
Bikestream
| project Street, ["Number of Empty Docks"] = No_Empty_Docks
| take 10
```

코드 설명:

- `["Number of Empty Docks"] = No_Empty_Docks: project` 연산자 내에서 열의 이름을 변경하는 방법입니다. 기존 열 이름인 `No_Empty_Docks`를 더 읽기 쉬운 `Number of Empty Docks`로 변경합니다. 공백이 포함된 이름은 따옴표 `" "`로 묶어줍니다.

KQL을 사용하여 데이터 요약하기

`summarize` 키워드를 함수와 함께 사용하여 데이터를 집계하고 조작할 수 있습니다.

1. `sum` 함수를 사용하여 렌탈 데이터를 요약하여 총 몇 대의 자전거가 사용 가능한지 확인하는 다음 쿼리를 시도해 보세요.

```
Bikestream
| summarize ["Total Number of Bikes"] = sum(No_Bikes)
```

코드 설명:

- `summarize`: 데이터를 집계하는 데 사용되는 핵심 연산자입니다.
- `["Total Number of Bikes"] = sum(No_Bikes):` `sum()`은 집계 함수로, `No_Bikes` 열의 모든 숫자 값을 더합니다. 그 결과를 `Total Number of Bikes`라는 새로운 이름의 열에 저장합니다. 이 쿼리는 테이블 전체의 자전거 총합을 계산합니다.

요약된 데이터를 지정된 열이나 표현식으로 그룹화할 수 있습니다.

2. 다음 쿼리를 실행하여 각 동네별로 사용 가능한 자전거 수를 확인하기 위해 자전거 수를 동네별로 그룹화합니다.

```
Bikestream
| summarize ["Total Number of Bikes"] = sum(No_Bikes) by Neighbourhood
| project Neighbourhood, ["Total Number of Bikes"]
```

코드 설명:

- `... by Neighbourhood:` `summarize` 연산자에 `by` 절을 추가하면, `Neighbourhood` 열의 고유한 값별로 그룹을 만들어 각 그룹에 대해 `sum(No_Bikes)` 집계를 수행합니다. SQL의 `GROUP BY`와 동일한 역할을 합니다.
- `| project ...:` `summarize`의 결과에서 원하는 열의 순서를 지정하거나 이름을 변경하기 위해 `project`를 다시 사용했습니다.

만약 자전거 정류장 중 동네에 대한 null 또는 빈 항목이 있는 경우, 요약 결과에는 분석에 좋지 않은 빈 값이 포함됩니다.

3. `case` 함수를 `isempty` 및 `isnull` 함수와 함께 사용하여 동네가 알려지지 않은 모든 여정을 후속 조치를 위해 **Unidentified** 카테고리 그룹화하도록 쿼리를 다음과 같이 수정합니다.

```
Bikestream
| summarize ["Total Number of Bikes"] = sum(No_Bikes) by Neighbourhood
| project Neighbourhood = case(isempty(Neighbourhood) or isnull(Neighbourhood),
"Unidentified", Neighbourhood), ["Total Number of Bikes"]
```

코드 설명:

- `case(condition, if_true, if_false)`: 조건에 따라 다른 값을 반환하는 함수입니다. SQL의 `CASE` 문과 유사합니다.
- `isempty(Neighbourhood)`: `Neighbourhood` 열의 값이 빈 문자열("")이면 `true`를 반환합니다.
- `isnull(Neighbourhood)`: `Neighbourhood` 열의 값이 `null`이면 `true`를 반환합니다.
- `... or ...`: 논리적 OR 연산자입니다.
- 전체 `case(...)` 표현식의 의미는 "만약 `Neighbourhood` 값이 비어 있거나 null이면 'Unidentified'라는 문자열을 반환하고, 그렇지 않으면 원래 `Neighbourhood` 값을 반환하라"입니다. 이를 통해 데이터 정제를 쿼리 내에서 수행할 수 있습니다.

Note: 이 샘플 데이터셋은 잘 관리되어 있으므로 쿼리 결과에 `Unidentified` 필드가 없을 수 있습니다.

KQL을 사용하여 데이터 정렬하기

데이터를 더 의미있게 만들기 위해 일반적으로 열을 기준으로 정렬하며, 이 과정은 KQL에서 `sort by` 또는 `order by` 연산자로 수행됩니다(동일하게 작동합니다).

1. 다음 쿼리를 시도해 보세요.

```
Bikestream
| summarize ["Total Number of Bikes"] = sum(No_Bikes) by Neighbourhood
| project Neighbourhood = case(isempty(Neighbourhood) or isnull(Neighbourhood),
"Unidentified", Neighbourhood), ["Total Number of Bikes"]
| sort by Neighbourhood asc
```

코드 설명:

- `sort by Neighbourhood asc`: 이전 단계의 결과를 `Neighbourhood` 열을 기준으로 오름차순(`asc`)으로 정렬합니다. 내림차순은 `desc`를 사용합니다.

2. 쿼리를 다음과 같이 수정하고 다시 실행하면 `order by` 연산자가 `sort by`와 동일하게 작동하는 것을 확인할 수 있습니다.

```
Bikestream
| summarize ["Total Number of Bikes"] = sum(No_Bikes) by Neighbourhood
| project Neighbourhood = case(isempty(Neighbourhood) or isnull(Neighbourhood),
"Unidentified", Neighbourhood), ["Total Number of Bikes"]
| order by Neighbourhood asc
```

KQL을 사용하여 데이터 필터링하기

KQL에서 `where` 절은 데이터를 필터링하는 데 사용됩니다. `where` 절에서 `and` 및 `or` 논리 연산자를 사용하여 조건을 결합할 수 있습니다.

1. 다음 쿼리를 실행하여 자전거 데이터를 필터링하여 첼시(Chelsea) 동네의 자전거 정류장만 포함하도록 합니다.

```
Bikestream
| where Neighbourhood == "Chelsea"
| summarize ["Total Number of Bikes"] = sum(No_Bikes) by Neighbourhood
| project Neighbourhood = case(isempty(Neighbourhood) or isnull(Neighbourhood),
"Unidentified", Neighbourhood), ["Total Number of Bikes"]
| sort by Neighbourhood asc
```

코드 설명:

- `where Neighbourhood == "Chelsea"`: `Bikestream` 테이블의 전체 행 중에서 `Neighbourhood` 열의 값이 "Chelsea"와 정확히 일치(`==`)하는 행만 남깁니다. 이 필터링된 데이터가 파이프를 통해 다음 `summarize` 연산자로 전달됩니다.

Transact-SQL을 사용하여 데이터 쿼리하기

개념 설명: KQL Database의 T-SQL 지원

`KQL Database`는 기본적으로 Transact-SQL(T-SQL)을 지원하지 않지만, Microsoft SQL Server를 에뮬레이션하는 **T-SQL 엔드포인트**를 제공하여 데이터에 T-SQL 쿼리를 실행할 수 있게 해줍니다. 이 기능은 KQL을 지원하지 않는 기존의 BI 도구나 애플리케이션(예: Tableau, Power BI의 일부 연결 모드, SSMS)이 `KQL Database`의 데이터에 접근할 수 있도록 하는 **호환성** 계층입니다.

중요한 제약 사항:

- 데이터 조회(`SELECT`)만 가능하며, 테이블 생성/수정/삭제(DDL)나 데이터 삽입/수정/삭제(DML)는 지원하지 않습니다.
- KQL과 호환되지 않는 일부 T-SQL 함수나 구문은 지원되지 않을 수 있습니다.
- 최상의 성능과 모든 기능을 활용하기 위해서는 `KQL Database`의 기본 쿼리 언어인 **KQL**을 사용하는 것이 강력히 권장됩니다.

Transact-SQL을 사용하여 테이블에서 데이터 검색하기

1. `queryset`에서 다음 Transact-SQL 쿼리를 추가하고 실행합니다.

```
SELECT TOP 100 * from Bikestream
```

- `SELECT TOP 100 *`: `Bikestream` 테이블에서 상위 100개 행의 모든 열(`*`)을 선택합니다. KQL의 `take 100`과 유사한 역할을 하지만, SQL에서는 정렬 순서가 지정되지 않으면 어떤 100개가 반환될지 보장되지 않습니다.

2. 쿼리를 다음과 같이 수정하여 특정 열을 검색합니다.

```
SELECT TOP 10 Street, No_Bikes
FROM Bikestream
```

- `SELECT TOP 10 Street, No_Bikes`: KQL의 `project Street, No_Bikes | take 10`과 기능적으로 동일합니다.

3. 쿼리를 수정하여 **No_Empty_Docks**를 더 사용자 친화적인 이름으로 바꾸는 별칭(alias)을 할당합니다.

```
SELECT TOP 10 Street, No_Empty_Docks as [Number of Empty Docks]
from Bikestream
```

코드 설명:

- `as [Number of Empty Docks]`: `No_Empty_Docks` 열의 별칭으로 `Number of Empty Docks`를 지정합니다. KQL의 `project ["Number of Empty Docks"] = No_Empty_Docks`와 동일한 기능입니다.

Transact-SQL을 사용하여 데이터 요약하기

1. 다음 쿼리를 실행하여 사용 가능한 총 자전거 수를 찾습니다.

```
SELECT sum(No_Bikes) AS [Total Number of Bikes]
FROM Bikestream
```

코드 설명:

- `SELECT sum(No_Bikes) ...`: 집계 함수 `sum()`을 사용하여 `No_Bikes` 열의 합계를 계산합니다. `AS` 키워드를 사용하여 결과 열의 이름을 지정합니다.

2. 쿼리를 수정하여 총 자전거 수를 동네별로 그룹화합니다.

```
SELECT Neighbourhood, Sum(No_Bikes) AS [Total Number of Bikes]
FROM Bikestream
GROUP BY Neighbourhood
```

코드 설명:

- `GROUP BY Neighbourhood`: `Neighbourhood` 열의 고유한 값별로 그룹을 만들어 각 그룹에 대해 `Sum(No_Bikes)`를 계산합니다. KQL의 `summarize ... by Neighbourhood`와 동일합니다.

3. 쿼리를 더 수정하여 `CASE` 문을 사용하여 알 수 없는 출처의 자전거 정류장을 후속 조치를 위해 **Unidentified** 카테고리 그룹화합니다.


```

SELECT CASE
    WHEN Neighbourhood IS NULL OR Neighbourhood = '' THEN 'Unidentified'
    ELSE Neighbourhood
END AS Neighbourhood,
SUM(No_Bikes) AS [Total Number of Bikes]
FROM Bikestream
GROUP BY CASE
    WHEN Neighbourhood IS NULL OR Neighbourhood = '' THEN 'Unidentified'
    ELSE Neighbourhood
END;

```

코드 설명:

- `CASE WHEN ... THEN ... ELSE ... END`: 조건부 로직을 구현합니다. `Neighbourhood`가 `NULL`이거나 빈 문자열('')이면 'Unidentified'를, 그렇지 않으면 원래 값을 반환합니다. KQL의 `case()` 함수와 동일한 역할을 합니다.
- `GROUP BY` 절에도 동일한 `CASE` 문을 사용하여 `SELECT` 목록과 일관되게 그룹화를 수행해야 합니다.

Transact-SQL을 사용하여 데이터 정렬하기

1. 다음 쿼리를 실행하여 그룹화된 결과를 동네별로 정렬합니다.

```

SELECT CASE
    WHEN Neighbourhood IS NULL OR Neighbourhood = '' THEN 'Unidentified'
    ELSE Neighbourhood
END AS Neighbourhood,
SUM(No_Bikes) AS [Total Number of Bikes]
FROM Bikestream
GROUP BY CASE
    WHEN Neighbourhood IS NULL OR Neighbourhood = '' THEN 'Unidentified'
    ELSE Neighbourhood
END
ORDER BY Neighbourhood ASC;

```

코드 설명:

- `ORDER BY Neighbourhood ASC`: 최종 결과를 `Neighbourhood` 열(CASE 문에 의해 변환된 결과)을 기준으로 오름차순(`ASC`)으로 정렬합니다. KQL의 `sort by` 또는 `order by`와 동일합니다.

Transact-SQL을 사용하여 데이터 필터링하기

1. 다음 쿼리를 실행하여 그룹화된 데이터를 필터링하여 동네가 "Chelsea"인 행만 결과에 포함되도록 합니다.

```

SELECT CASE
    WHEN Neighbourhood IS NULL OR Neighbourhood = '' THEN 'Unidentified'
    ELSE Neighbourhood
END AS Neighbourhood,
SUM(No_Bikes) AS [Total Number of Bikes]
FROM Bikestream
GROUP BY CASE
    WHEN Neighbourhood IS NULL OR Neighbourhood = '' THEN 'Unidentified'
    ELSE Neighbourhood
END
HAVING Neighbourhood = 'Chelsea'
ORDER BY Neighbourhood ASC;

```

코드 설명:

- `HAVING Neighbourhood = 'Chelsea'`: `GROUP BY`를 통해 집계된 결과에 대한 필터링을 수행합니다.
`WHERE` 절은 그룹화하기 전에 개별 행을 필터링하는 반면, `HAVING` 절은 그룹화한 후에 그룹을 필터링합니다.
 이 경우, `GROUP BY`의 결과 중 `Neighbourhood`가 'Chelsea'인 그룹만 선택합니다.

리소스 정리

이 실습에서는 `eventhouse`를 생성하고 KQL과 SQL을 사용하여 데이터를 쿼리했습니다.

`KQL Database` 탐색을 마쳤다면, 이 실습을 위해 생성한 Workspace를 삭제할 수 있습니다.

1. 왼쪽 바에서 Workspace 아이콘을 선택합니다.
2. 도구 모음에서 **Workspace settings**를 선택합니다.
3. **General** 섹션에서 **Remove this workspace**를 선택합니다.